

TCS-504 — System Design — Assignment 1

Movie Ticket Booking System

Step A — Requirement Analysis

Functional Requirements

FR1 — Listing movies: the system displays every movie currently playing, showing at least title, language, and duration, before the customer picks one.

FR2 — Listing shows: once a movie is chosen, the system lists every show of that movie, showing screen name and start time for each.

FR3 — Seat layout: once a show is chosen, the system displays every seat of that show's screen grouped by type, marked AVAILABLE or BOOKED, and this view must never mutate any seat's status.

FR4 (*given*) — Booking: a customer selects one or more seat numbers for a show. If **any** selected seat is already BOOKED, the whole booking is rejected and no seat changes state. Booking is confirmed **only after** payment succeeds.

FR5 — Pricing: the total for a booking equals the sum of each selected seat's price by type (SILVER ₹150, GOLD ₹250, PLATINUM ₹400), computed before payment is attempted and shown to the customer.

FR6 (*given*) — Payment: exactly one method (UPI / Card / Cash) per booking. If payment fails, seats are released and booking status becomes FAILED.

FR7 — Ticket printing: on a CONFIRMED booking only, the system prints a ticket containing booking id, movie title, screen, show time, all booked seat numbers, and total amount.

FR8 — Cancellation: given a valid, CONFIRMED booking id, the system sets its status to CANCELLED and every seat that booking held returns to AVAILABLE; cancelling an unknown or non-CONFIRMED booking id must show a clear error and change nothing.

Non-Functional Requirements

NFR1 — Modularity: one class per file; a class must not directly manipulate another class's private data (e.g. BookingService changes seat status only through ShowSeat::bookSeat(), never by reaching into its fields).

NFR2 — Extensibility: adding a new payment method (e.g. NetBanking) must require writing one new class that extends Payment and must not require editing BookingService, Booking, or the menu.

NFR3 — Input validation: every console input (menu choice, seat number, movie/show index) is validated; invalid input produces a clear message and returns to the menu without crashing or corrupting state.

NFR4 — Consistency: a booking's seats and its payment status change together — a seat is never left BOOKED for a FAILED or CANCELLED booking (no partial/inconsistent state is ever visible between operations).

Step B — Noun–Verb Analysis

Noun found	Keep as a class?	Reason
Movie	Yes	has its own data and identity (title, language, duration)
Seat	Yes	has number, type, price — a physical thing on the screen
Screen	Yes	has an identity (screen number) and owns seats
Cinema	Yes	the theatre; owns screens; has a name
Show	Yes	a screening event with its own identity (movie + screen + time)
“seat layout”	No	it is a <i>view</i> of a Show's seats, not a thing → a print method on Show
Customer	Yes	has name and phone; identity independent of any one booking
Booking	Yes	has its own id, seats, amount and status — a transaction record
Ticket	No	it is the <i>printed output</i> of a Booking, not a stored thing → TicketPrinter.printTicket()
Payment	Yes	represents the act of paying; varies by method → abstract class + subtypes
UPI / Card / Cash	Yes (as subclasses)	each pays differently → concrete Payment subclasses

“seat status” (AVAILABLE/BOOKED)	No, not on Seat	status differs <i>per show</i> for the same physical seat → its own class ShowSeat
Price	No	not a standalone thing, just a number derived from seat type → a method in PriceCalculator
“book”, “pay”, “cancel”, “print” (verbs)	—	become methods: bookSeat(), pay(), cancelBooking(), printTicket()

Key verbs identified: **see** (list movies/shows/seats), **pick** (choose show/seats), **book**, **pay**, **print**, **cancel** — each maps directly to a method on the responsible class above.

Step C — Classes: knows / does / must NOT do

Core entity classes

Class	Knows	Does	Must NOT do
Movie	title, language,	returns its own fields duration	know about screens, shows, or price
Seat	seat number, seat type	returns its own fields	store or know booking status
Screen	screen name, its list of builds/returns its seat layout	Seat	know about movies, shows, or bookings
Cinema	cinema name, its list of adds/returns screens	Screen	manage shows, bookings, or payments
Show	which Movie, which Screen, start time, its ShowSeats	finds a seat by number, prints the seat layout	know about payments or bookings
ShowSeat	one Seat + its status this show	books/releases itself (with a for rejection rule)	calculate price or print tickets
Customer	name, phone	returns its own fields	hold or own bookings itself
Booking	id, show, customer, amount, status	changes its own status seats, (confirmed/failed/cancelled)	print tickets or run the booking flow

Behaviour / service classes

Class	Knows	Does	Must NOT do
Payment (abstract)	nothing (contract only)	declares pay(amount) as pure virtual	know about Booking, Show, or seats
UpiPayment / CardPayment / CashPayment	how that method simulates success/failure	implements pay(amount)	know about bookings or seats
Class	Knows	Does	Must NOT do
PriceCalculator	the three seattype prices	totals a seat or list of seats	touch booking status or payment
TicketPrinter	nothing (stateless)	formats and prints a ticket	change booking or seat status (no side effects)
BookingService	its own PriceCalculator, book→price→pay→confirm/fail→print, TicketPrinter, all and cancel bookings made	runs	draw the console menu or format ticket text
Cinema/main menu	which Cinema, BookingService to call	reads console input, calls the right method	contain booking or pricing logic itself

Step D — Relationships

For every pair, the choice is justified with the **lifetime test**: *if the whole is destroyed, does the part die with it?*

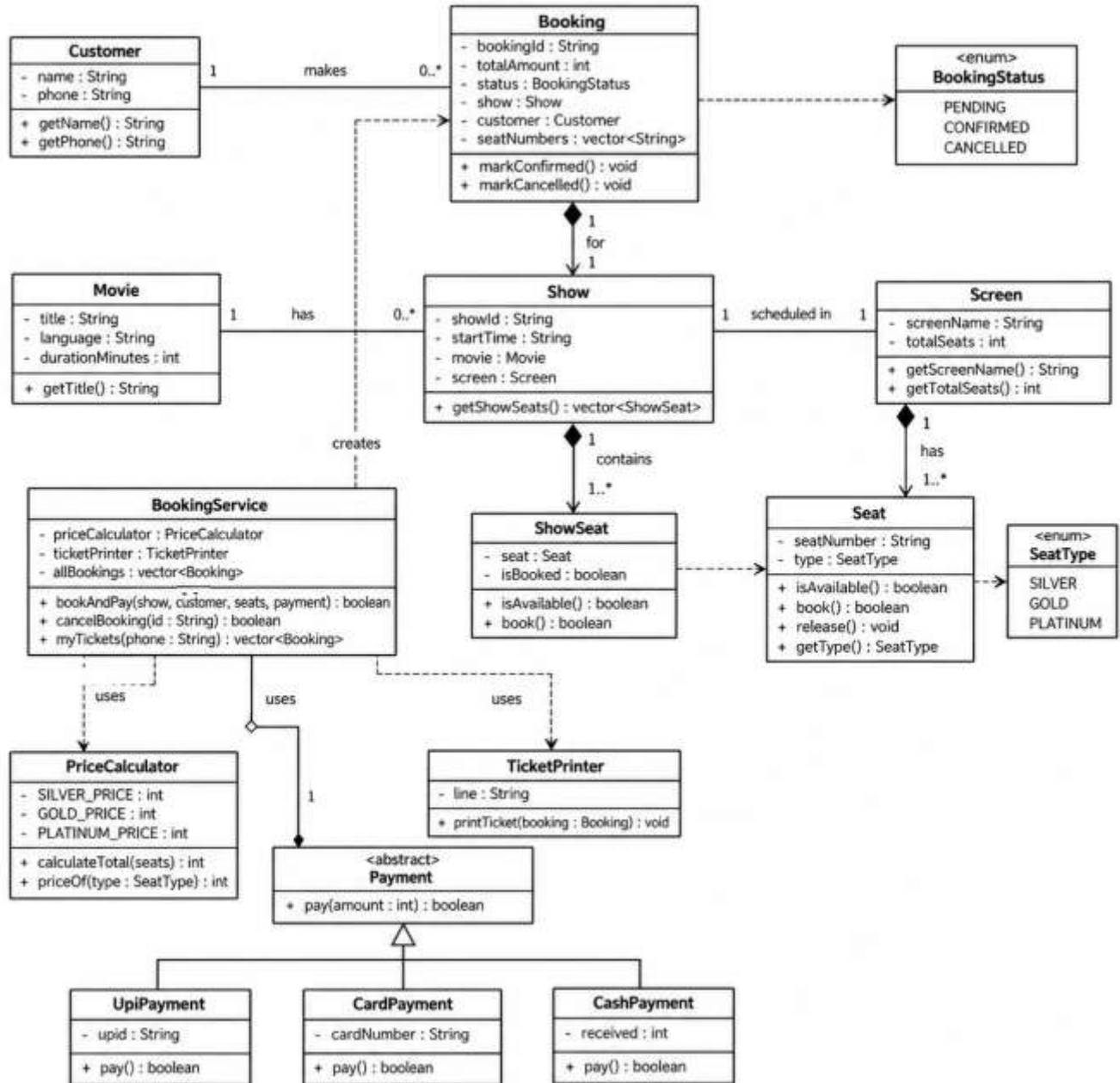
Pair	Choice	Justification
Cinema — Screen	Composition (◆)	A Screen has no meaning outside its Cinema; destroy the Cinema object and its Screens are destroyed with it (they are stored by value inside Cinema).
Screen — Seat	Composition (◆)	A physical Seat belongs to exactly one Screen; a Seat cannot be moved to another screen or exist without one — destroying the Screen destroys its Seats.
Show — Movie	Aggregation (◇) (given)	A Show <i>borrow</i> s a Movie. Cancel the 6 PM show and “3 Idiots” still exists and still plays at 9 PM — the Show stores a pointer and never deletes the Movie.
Pair	Choice	Justification
Show — Screen	Aggregation (◇)	A Show <i>borrow</i> s a Screen for its duration. Delete the 6 PM Show and Screen-1 still exists and can host the 9 PM Show — the Show never owns or deletes the Screen.
Show — ShowSeat	Composition (◆)	ShowSeat objects exist only to represent <i>this</i> Show’s seat statuses; if the Show is deleted, its ShowSeats are meaningless and are destroyed with it (they are created and stored by the Show itself).
Booking — Customer	Association (→)	A Booking refers to a Customer but neither owns the other’s lifetime — the same Customer can exist with zero, one, or many Bookings, and deleting a Booking does not delete the Customer.
Booking — ShowSeat	Aggregation (◇)	A Booking refers to existing ShowSeats (owned by the Show); cancelling/deleting the Booking must NOT delete the ShowSeat objects — it only flips their status back to AVAILABLE.
Booking — Payment	Association (→)	A Booking is linked to the Payment attempt used to pay for it, but the Payment object’s lifetime is independent — it is created by main()/caller and can be discarded right after pay() returns, without affecting the Booking record.

Payment — UpiPayment	Inheritance (D)	UpiPayment is a Payment — it inherits and implements the abstract pay() contract; this is a type relationship, not an ownership one, so the lifetime test does not apply the same way (a subclass IS the class, extended).
BookingService — Booking	Aggregation/creates (◇)	BookingService creates each Booking (via new) and keeps it in its own list for cancellation/lookup — in this design BookingService does own the memory (deletes them in its destructor), but a Booking's <i>meaning</i> is independent of the service; conceptually it is closer to aggregation since a Booking record is meaningful even if the service were swapped out.

Step E — Class Diagram

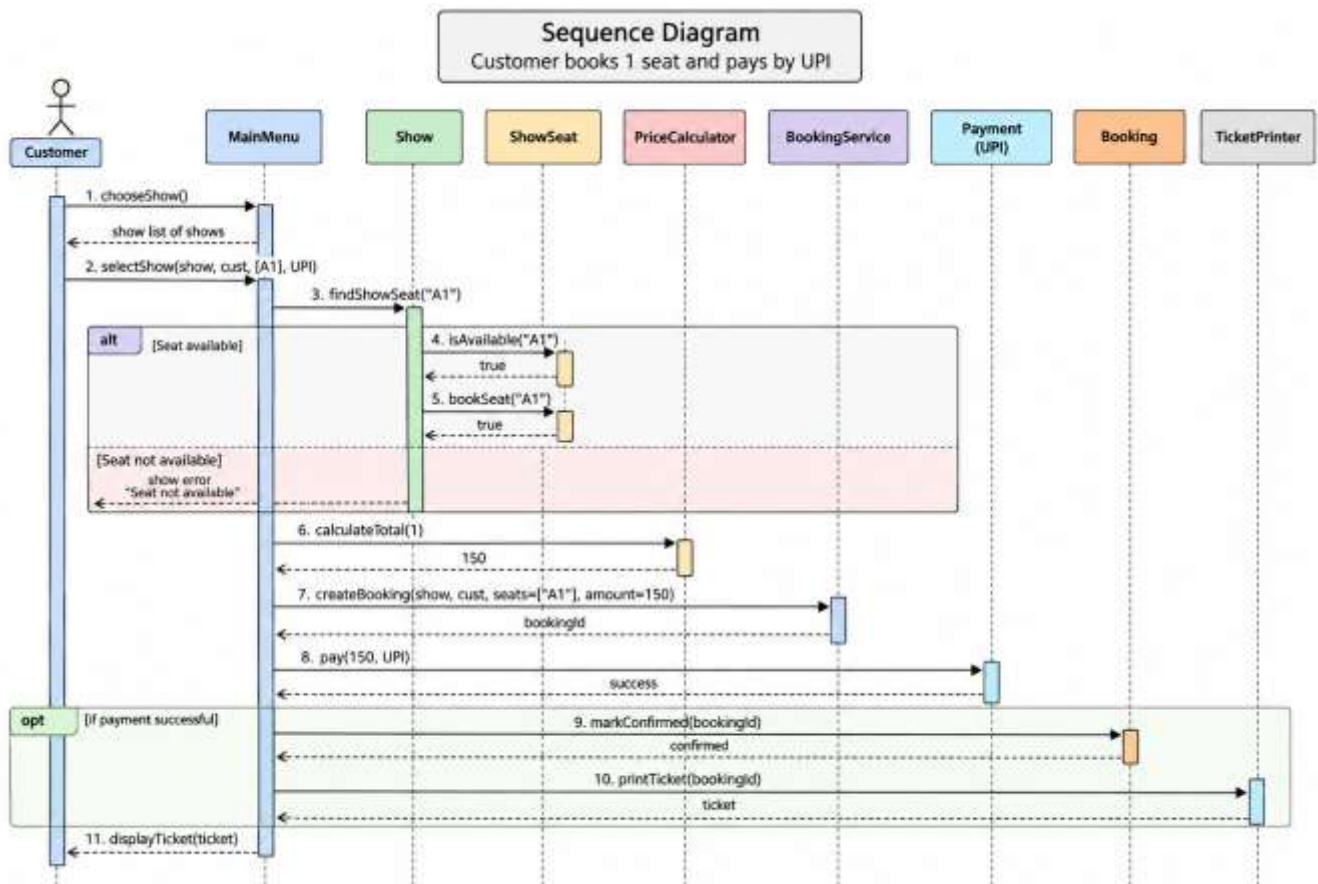
The diagram below contains every class from Step C in one page, with three compartments (name / attributes / methods), correct UML notation for each relationship type, and multiplicities. Payment is marked <<abstr

Movie Ticket Booking System – Class Diagram



Step F — Sequence Diagram

Use case: “**customer books 1 seat and pays by UPI**”, from choosing a show to the ticket being printed. Lifelines: customer, mainMenu (console), bookingService, show, showSeat, priceCalculator, payment, booking, ticketPrinter. Activation bars show when each object is busy; dashed arrows are return messages; <<create>> marks object creation.



Sequence Diagram

Step G — Modular Code

Implemented in C++, one class per file (course rule), no separate header files (see the deliberate design trade-off noted under Step H). File structure:

01_Movie.cpp	06_ShowSeat.cpp	11_PriceCalculator.cpp
02_Seat.cpp	07_Customer.cpp	12_TicketPrinter.cpp
03_Screen.cpp	08_Booking.cpp	13_BookingService.cpp
04_Cinema.cpp	09_Payment.cpp	main.cpp
05_Show.cpp	10_PaymentTypes.cpp	

Compiled with: `g++ -std=c++17 -Wall -o ticket_booking main.cpp`. A full demo run (listing movies, listing shows, seat layout, booking + UPI payment + ticket, an invalid menu choice, and an already-booked/invalid-seat rejection) is captured in `demo_output.txt`.

Edge cases verified (all four):

1. Booking an already-BOOKED seat → rejected, no state change (all seats are validated *before* any of them are booked).
2. A failed payment → booking marked FAILED, seats released back to AVAILABLE (verified with a forced-failure UpiPayment).
3. Cancelling a booking → its seats return to AVAILABLE (shown live in the transcript: BK1001 cancelled, seat layout shows [] again).
4. An invalid seat number or menu choice → a clear message is printed and the loop continues, no crash (shown live: menu option 99, seat ZZ).

OOP concepts and clean-code rules are called out with inline `//` comments in the source.
Highlights:

- **Encapsulation:** all fields private; state changes only through methods like `ShowSeat::bookSeat()`.
- **Abstraction:** Payment is a pure-virtual abstract class — only the contract, no logic.
- **Inheritance / Runtime polymorphism:** `UpiPayment`, `CardPayment`, `CashPayment` extend `Payment`; `BookingService` calls `payment.pay(total)` through a `Payment&` and the correct override runs.
- **Compile-time polymorphism:** overloaded constructors (`Movie()` vs `Movie(title, language, duration)`) and overloaded `PriceCalculator::calculateTotal(...)` (single seat vs. a vector of seats).
- **Static members:** `Booking::nextBookingSequence` generates unique, incrementing booking IDs shared across all `Booking` objects.
- **this keyword:** used throughout constructors to disambiguate member fields from parameters of the same name.
- **Constants over magic numbers:** `PriceCalculator::SILVER_PRICE/GOLD_PRICE/PLATINUM_PRICE`, never bare `150/250/400`.

Step H — SOLID Mapping and Deliberate Omission

Principle	Where it is applied
-----------	---------------------

S — Single Responsibility	<code>Booking</code> only <i>holds</i> booking data; it does not print tickets (that's <code>TicketPrinter</code>) or run the flow (that's <code>BookingService</code>). A pricing rule change touches only <code>PriceCalculator</code> ; a layout/format change touches only <code>TicketPrinter</code> or <code>Show::printSeatLayout()</code> — never both from the same edit.
----------------------------------	--

O — Open/Closed Principle	<code>BookingService.bookAndPay()</code> takes a <code>Payment&</code> . Adding <code>NetBankingPayment</code> means writing one new subclass of <code>Payment</code> —
Principle	Where it is applied

	BookingService is never edited or recompiled for new payment types.
L — Liskov Substitution	Every Payment subclass (UpiPayment, CardPayment, CashPayment) is usable through a plain Payment& with no extra setup call and the same contract (pay(amount) returns bool) — BookingService never checks <i>which</i> subclass it received.
D — Dependency Inversion	BookingService depends on the abstract Payment, not on any concrete payment class; the concrete object is constructed by main() and passed in by reference — high-level orchestration logic never depends on low-level payment details.
<i>(I — Interface Segregation, considered)</i>	Payment only demands pay() — it deliberately does not force a refund() method on every subtype, since not all payment methods in scope support refunds; adding refunds later would need a separate, smaller interface rather than bloating Payment.

One thing deliberately NOT done

We deliberately did **not** implement proper separate compilation with .h header files and a build system (Makefile/CMake) — the assignment’s “no header files” rule was met instead by writing each class as a single, include-guarded .cpp file and pulling them together with #include inside main.cpp. This keeps “one class per file” as the organizing structure, but it means the project compiles as one large translation unit rather than being compiled and linked as independent object files. In a production system this would be replaced by proper header/implementation separation so each class could be compiled independently and reused across binaries.

We also deliberately did **not** add persistence (a database or file storage) — all data (movies, shows, bookings) lives in memory and is lost when the program exits, since the assignment scope is a single console session, not multi-day booking retention.